สมาชิก

673380056-7 รพีภัทร แก้วคูณ (ไผ่) sec.2

673380068-0 สุชาดา ศรีจักรโคตร (เค้ก) sec.2

673380286-0 พิไลพร คำเวียง (เอวา) sec.2

673380297-5 หทัยพัทธ วิสุทธิธรรม (แก้ม) sec.1

673380299-1 เจตนิพิฐ แก้วทรัพย์ (ลูกแก้ว) sec.1

Excellent. We will now **strictly use your new topology and configuration**:

- ✅ R1 G0/0 → Internet (DHCP, NAT outside)
- ✅ R1–R2 connected via **Serial S0/0/0**
- ✅ ISP domain: **100.10.10.0/30**
- ✅ LAN A: 192.168.10.0/24
- ✅ LAN B: 192.168.20.0/24
- ✅ Microservices from **Week03 (Phase 1 Mockup)** FastAPI architecture

We now reframe this as:

🧪 LAB 5 — Internet Edge + ISP Serial WAN + Week03 Microservices

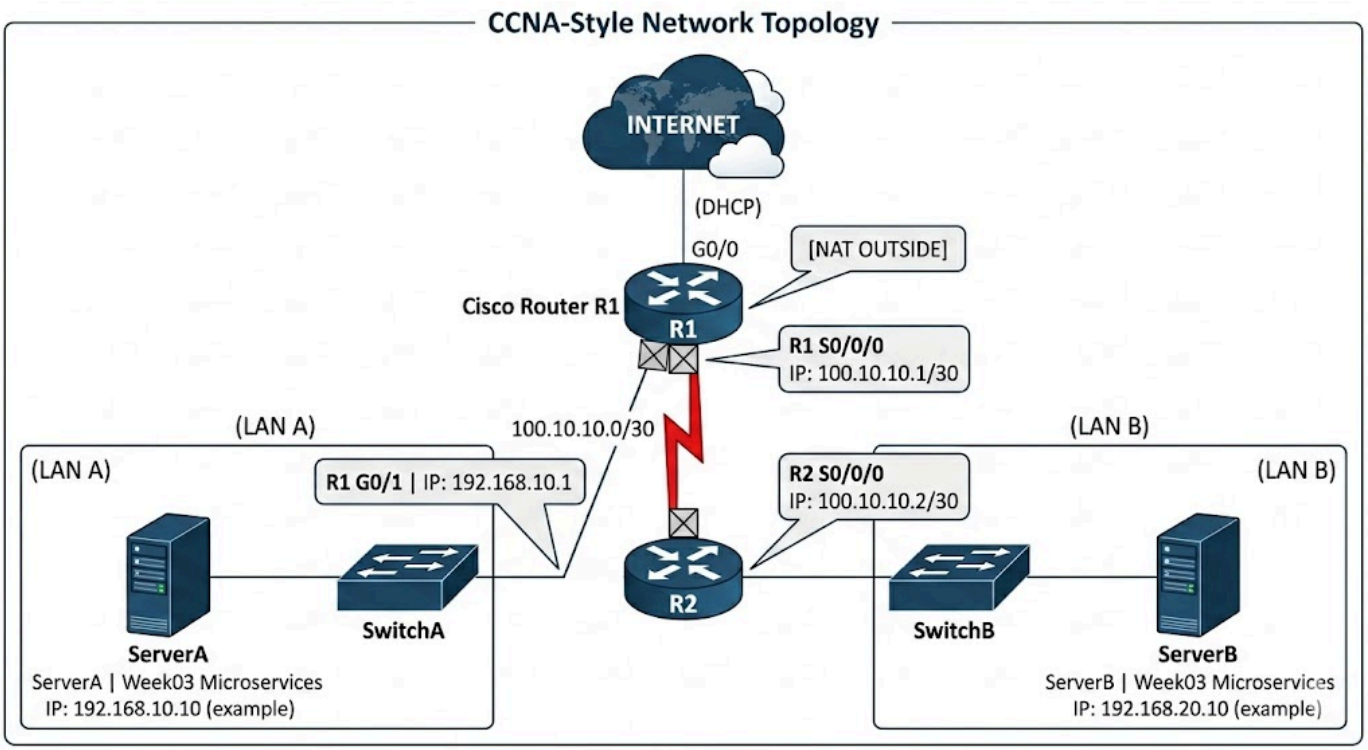
🎯 LAB OBJECTIVE

Transform Week03 microservices into a **realistic enterprise deployment**:

- R1 = Internet Edge Router (NAT + DHCP WAN)
- Serial WAN = ISP Domain (100.10.10.0/30)
- R2 = Branch Router
- Microservices deployed on ServerA and ServerB
- Validate cross-site microservice access over routed WAN + NAT

🗺️ LAB 5 TOPOLOGY (Reframed)

```
None
                    INTERNET
                       |
                   (DHCP)
                    R1 G0/0
                       |
                [NAT OUTSIDE]
                       |
                R1 S0/0/0
             100.10.10.1/30
                       |
                R2 S0/0/0
             100.10.10.2/30
                       |
           -----------------------
           |                      |
R1 G0/1 (LAN A)        R2 G0/1 (LAN B)
192.168.10.1            192.168.20.1
           |                      |
     ServerA                ServerB
Week03 Microservices   Week03 Microservices
```



📍 ADDRESS TABLE (FINAL)

Device	Interface	IP	Role
R1	G0/0	DHCP	Internet
R1	S0/0/0	100.10.10.1/30	ISP
R1	G0/1	192.168.10.1/24	LAN A
R2	S0/0/0	100.10.10.2/30	ISP
R2	G0/1	192.168.20.1/24	LAN B
ServerA	eth0	192.168.10.10	Microservices A
ServerB	eth0	192.168.20.10	Microservices B

🔧 PART 1 — CISCO CONFIGURATION (STRICT)

🧱 R1 — Internet Edge Router

```
None
enable
conf t
hostname R1

interface g0/0
 ip address dhcp
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

interface s0/0/0
```

```
 ip address 100.10.10.1 255.255.255.252
 clock rate 64000
 no shutdown

ip route 192.168.20.0 255.255.255.0 100.10.10.2
ip route 0.0.0.0 0.0.0.0 g0/0

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

```
Router(config)#hostname R1
R1(config)#interface g0/0
R1(config-if)#ip address dhcp
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

R1(config-if)#exit
R1(config)#interface g0/1
R1(config-if)#ip address 192.168.10.1 255.255.255.0
R1(config-if)#ip nat outside
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R1(config)#interface s0/0/0
R1(config-if)#ip address 100.10.10.1 255.255.255.252
R1(config-if)#clock rate 64000
R1(config-if)#no shutdown

%LINK-5-CHANGED: Interface Serial0/0/0, changed state to down

R1(config)#ip route 192.168.20.0 255.255.255.0 100.10.10.2
R1(config)#ip route 0.0.0.0 0.0.0.0 g0/0
```

g0/0: เปิดใช้งาน รับ IP อัตโนมัติ (DHCP) และกำหนดเป็น NAT Outside สำหรับออก Internet
g0/1: เปิดใช้งานและตั้ง IP เป็น 192.168.10.1 สำหรับเครือข่าย LAN A
s0/0/0: เปิดใช้งาน ตั้ง IP เป็น 100.10.10.1 และกำหนด clock rate เพื่อเชื่อมต่อกับ R2
ไปหา ServerB: สั่งให้ R1 รู้ว่าถ้าจะไปวง 192.168.20.0 ให้ส่งข้อมูลไปที่ 100.10.10.2 (R2)

Default Route: ถ้าไม่รู้ปลายทาง ให้ส่งข้อมูลออก Internet ทาง g0/0

🧱 R2 — Branch Router

```
None
enable
conf t
hostname R2

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 no shutdown

interface s0/0/0
 ip address 100.10.10.2 255.255.255.252
 no shutdown

ip route 192.168.10.0 255.255.255.0 100.10.10.1
ip route 0.0.0.0 0.0.0.0 100.10.10.1

end
write memory
```

```
Router(config)#hostname R2
R2(config)#interface g0/1
R2(config-if)#ip address 192.168.20.1 255.255.255.0
R2(config-if)#no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

R2(config)#interface s0/0/0
R2(config-if)#ip address 100.10.10.2 255.255.255.252
R2(config-if)#no shutdown

R2(config)#ip route 192.168.10.0 255.255.255.0 100.10.10.1
R2(config)#ip route 0.0.0.0 0.0.0.0 100.10.10.1
R2(config)#end

R2#write memory
Building configuration...
[OK]
```

g0/1: เปิดใช้งานและตั้ง IP เป็น 192.168.20.1
s0/0/0: เปิดใช้งานและตั้ง IP เป็น 100.10.10.2

ไปหา **Server 1:** สั่งให้ R2 รู้ว่าถ้าจะไปวง 192.168.10.0 ให้โยนข้อมูลไปที่ 100.10.10.1 (R1)
ไปหา **Internet:** ตั้ง Default Route (0.0.0.0) ให้วิ่งไปที่ R1 เช่นกัน

💻 PART 2 — Deploy Week03 Microservices

Based on your Week03 architecture from `PROJECT_PROFILE.md`

We deploy:

- Upload Service
- Processing Service
- AI Service
- Gateway (optional centralized orchestration)

🛠️ On ServerA (LAN A)

```shell
cd automation/week03-microservices/phase1
python start_services.py
```

Services:

- Upload → 8000
- Processing → 8001
- AI → 8002
- Gateway → 9000

ServerA IP:

```none
192.168.10.10
```

```
📊 Service Status:
----------------------------------------
INFO:     127.0.0.1:58754 - "GET /health HTTP/1.1" 200 OK
Upload Service        🟢 Running
INFO:     127.0.0.1:64249 - "GET /health HTTP/1.1" 200 OK
Processing Service    🟢 Running
INFO:     127.0.0.1:64252 - "GET /health HTTP/1.1" 200 OK
AI Service            🟢 Running
INFO:     127.0.0.1:64255 - "GET /health HTTP/1.1" 200 OK
INFO:httpx:HTTP Request: GET http://127.0.0.1:8000/health "HTTP/1.1 200 OK"
INFO:     127.0.0.1:64256 - "GET /health HTTP/1.1" 200 OK
INFO:httpx:HTTP Request: GET http://127.0.0.1:8001/health "HTTP/1.1 200 OK"
INFO:     127.0.0.1:64257 - "GET /health HTTP/1.1" 200 OK
INFO:httpx:HTTP Request: GET http://127.0.0.1:8002/health "HTTP/1.1 200 OK"
INFO:     127.0.0.1:64254 - "GET /health HTTP/1.1" 200 OK
Gateway Service       🟢 Running
```

จากผลลัพธ์ข้างต้น แสดงให้เห็นว่า ระบบ Microsevices บน Server A ถูก deploy สำเร็จ โดยทุก service ได้แก่ Upload, Processing, AI, และ Gateway อยู่ในสถานะ running นั่นคือ สามารถทำงานได้ปกติ และสามารถตรวจสอบสถานะผ่าน endpoint /health ได้

🛠️ On ServerB (LAN B)

Same deployment:

```shell
python start_services.py
```

ServerB IP:

```none
192.168.20.10
```

```
🔄 Press Ctrl+C to stop all services...

📊 Service Status:
----------------------------------------
Upload Service        🟢 Running
Processing Service    🟢 Running
AI Service            🟢 Running
Gateway Service       🟢 Running

📊 Service Status:
----------------------------------------
Upload Service        🟢 Running
Processing Service    🟢 Running
AI Service            🟢 Running
Gateway Service       🟢 Running
```

จากผลลัพธ์ข้างต้น แสดงให้เห็นว่า ระบบ Microsevices บน Server B ถูก deploy สำเร็จ โดยทุก service ได้แก่ Upload, Processing, AI, และ Gateway อยู่ในสถานะ running นั่นคือ สามารถทำงานได้ปกติ และสามารถตรวจสอบสถานะผ่าน endpoint /health ได้

🧪 PART 3 — WAN + INTERNET TESTING

🔵 Test 1 — Cross-Site Upload (LAN A → LAN B)

From ClientA:

```
None
curl http://192.168.20.10:8000/health
```

Should traverse:

ClientA → R1 → Serial ISP → R2 → ServerB

```
StatusCode        : 200
StatusDescription : OK
Content           : {"status":"healthy","service":"upload-service","timestamp":"2026-03-31T13:45:51.357594"}
RawContent        : HTTP/1.1 200 OK
                    Content-Length: 88
                    Content-Type: application/json
                    Date: Tue, 31 Mar 2026 06:45:50 GMT
                    Server: uvicorn

                    {"status":"healthy","service":"upload-service","timestamp":"2026-03-31T13:4...
Forms             : {}
Headers           : {[Content-Length, 88], [Content-Type, application/json], [Date, Tue, 31 Mar 2026 06:45:50 GMT], [Server, uvicorn]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 88
```

จากผลลัพธ์ข้างต้น แสดงให้เห็นว่า

1. การเชื่อมต่อระหว่าง LAN A และ LAN B ข้าม Serail WAN ทำงานได้อย่างถูกต้อง
2. Static route บน R1 และ R2 ทำงานได้ตามปกติ
3. Microservice ส่วน Upload service บน Server B ทำงานปกติและสามารถตอบสนองต่อการตรวจสอบสถานะจากอีกไซต์ได้

กล่าวโดยสรุปคือ ระบบสามารถรองรับการเรียกใช้ mocroservice ที่อยู่คนละเครือข่าย ผ่าน WAN ที่มีการกำหนดเส้นทางแบบ Static routing โดยไม่มี NAT ระหว่าง R1 และ R2

🔵 Test 2 — Full Workflow Across ISP

From LAN A client:

```
None
curl -X POST http://192.168.20.10:9000/process-file
```

Flow:

Upload → Processing → AI → Response returned across WAN

```
{"workflow_id":"workflow_20260331_140244","file_id":"95700e95-6e84-4c85-a61b-6efd19aa25f5","upload_status":"completed","processing_status":"completed",
"ai_analysis_status":"completed","total_time":7.103029,"timestamp":"2026-03-31T14:02:51.685591"}
```

จากผลลัพธ์ข้างต้น แสดงให้เห็นว่าระบบสามารถประมวลผลครบทั้ง 3 ขั้นตอน ได้แก่ Upload Processing AI ได้สำเร็จ โดยใช้เวลา 7.10 วินาที
โดยเครือข่าย WAN ข้าม Serail ISP ระหว่าง R1 และ R2 ทำงานได้ และรองรับการเรียกใช้บริการแบบกระจายศูนย์ข้าม LAN ได้

🔵 Test 3 — Internet Reachability

From ServerA:

```
None
ping 8.8.8.8
```

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=13ms TTL=113
Reply from 8.8.8.8: bytes=32 time=13ms TTL=113
Reply from 8.8.8.8: bytes=32 time=15ms TTL=113
Reply from 8.8.8.8: bytes=32 time=12ms TTL=113

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 12ms, Maximum = 15ms, Average = 13ms
```

จากผลลัพธ์ข้างต้น แสดงให้เห็นว่า การทดสอบ ping จาก Server A ไปยัง 8.8.8.8 พบว่าสามารถ ping ได้สำเร็จ ซึ่งยืนยันได้ว่า Server A ใน LAN A สามารถเข้าถึงเครือข่ายอินเทอร์เน็ตได้ผ่าน R1 ที่ทำหน้าที่ NAT Overload แม้ว่าการรับส่งข้อมูลผ่านเส้นทาง Serial WAN ผ่าน R2 ก่อนถึง R1
สรุปได้ว่า ระบบการกำหนดเส้นทางแบบ Static Routing และการแปลงที่อยู่เครือข่ายทำงานได้อย่างถูกต้อง

Verify NAT overload working on R1:

```
None
show ip nat translations
```

🔎 Verification Commands

On R1

```
None
show ip route
show ip nat translations
show ip interface brief
```

On R2

```
None
show ip route
ping 100.10.10.1
```

สรุปผลการทดลอง LAB 5

ในการทดลองนี้ได้นำสถาปัตยกรรม Microservices จาก Week03 มาปรับใช้ร่วมกับโครงสร้างเครือข่ายแบบองค์กร โดยมี Router สองตัวเชื่อมต่อกันผ่าน Serial WAN ในโดเมน ISP ที่ใช้หมายเลขเครือข่าย 100.10.10.0/30 Router R1 ทำหน้าที่เป็น Internet Edge Router เชื่อมต่อกับอินเทอร์เน็ตผ่าน DHCP และทำ NAT Overload ส่วน Router R2 ทำหน้าที่เป็น Branch Router เชื่อมต่อกับ LAN B
และได้ทำการติดตั้ง Microservices ทั้งสี่ตัว ได้แก่ Upload Service, Processing Service, AI Service และ Gateway Service บน ServerA ใน LAN A และ ServerB ใน LAN B ซึ่งทั้งสองเครื่องสามารถรัน service

จากนั้นได้ทำการทดสอบ 3 ส่วน
ส่วนแรก ทดสอบการเชื่อมต่อข้ามไซต์ด้วยการเรียกดูสถานะของ Upload Service บน ServerB จากเครื่องใน LAN A ผลการทดสอบพบว่าการเชื่อมต่อผ่าน Serial WAN ทำงานได้อย่างถูกต้อง โดย Static Route บน R1 และ R2 สามารถส่ง request จาก LAN A ไปยัง LAN B ได้สำเร็จ

ส่วนที่สอง ทดสอบการทำงานแบบครบวงจร โดยส่งคำขอประมวลผลไฟล์ไปยัง Gateway Service บน ServerB ซึ่งจะเรียกใช้บริการ Upload, Processing และ AI ตามลำดับ ผลการทดสอบพบว่าระบบสามารถประมวลผลครบทั้งสามขั้นตอนได้ภายในเวลา 7.10 วินาที แสดงให้เห็นว่าเครือข่าย WAN สามารถรองรับการเรียกใช้บริการแบบกระจายศูนย์ข้าม LAN ได้

ส่วนที่สาม ทดสอบการเข้าถึงอินเทอร์เน็ต โดยให้ ServerA ใน LAN A ส่ง ping ไปยัง 8.8.8.8 ผลการทดสอบพบว่าสามารถติดต่อได้สำเร็จ แสดงให้เห็นว่า R1 สามารถทำ NAT Overload ได้อย่างถูกต้อง แม้ว่า traffic จะต้องวิ่งผ่าน Serial WAN ไปยัง R2 ก่อนแล้วจึงไป R1 ก่อนออกสู่อินเทอร์เน็ต

กล่าวโดยสรุป การทดลอง LAB 5 Microservices สามารถทำงานในสภาพแวดล้อมเครือข่ายระดับองค์กรที่มีการเชื่อมต่อ WAN และ NAT ได้อย่างมีประสิทธิภาพ